Final Exam (Part 1)

Program Design and Data Structures (1DL201)

Teachers: Johannes Borgström and Eva Darulova

2023-06-09 / 08:00 - 12:00

Instructions

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Write your **exam code** at the top right corner of **each page**.
- There are **16 questions**. Each question can award up to 3 points. You can obtain a maximum of **36 points**.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function documentation if the question text states so.
- Write your answers directly on the question sheet. You may use English and Swedish.
- Eva or Johannes will come to the exam hall around one hour after the beginning of the exam to answer questions.

Question 1: Code for Communication (2 points)

Fill in the expression to return, such that

- if `x` and `y` are both false, the function `question1` should call the function `not_fun()` and return the result.
- otherwise, `question1` should call the function `fun()` and return the result.

You may **not** use any unary or binary Boolean operators (`!`, `||`, `&&`), but you should use the ternary conditional operator (`? :`).

```
function question1(x, y) {
    return




}
```

```
function question1(x, y) {
    return x ? fun() : y ? fun() : not_fun();
}
```

Question 2: Recursion (2 points)

Consider the following function.

```
  function hyper(n, a, b) {
    return n === 0 ? b + 1
         : b === 0 ? ( n === 1 ? a
                     : n === 2 ? 0
                     : 1)
         : hyper(n - 1, a, hyper(n, a, b-1));
  }
```

Does it define an iterative or a recursive process? Motivate your answer.

Answer:

Recursive, since the final line is reachable and has two calls to the `hyper` function.

Question 3: Scope (1 point)

What is the result of evaluating the following Source program?

```
function g(x) {
    function g(x) {
        function g(x) {
            return (x <= 3) ? 34 : g(x - 3);
        }
        return (x <= 2) ? 23 : g(x - 2);
    }
    return (x <= 1) ? 12 : g(x - 1);
}
g(100);
```

Answer: 34

Question 4: Higher-order programming (2 points)

Rewrite the following code such that it does **not** use any defined or anonymous functions (lambda expressions), but computes the same result.

```
function g(f) {
    return x => f(f(x));
}
g(x => 2 * x + 1)(3);
```

Your solution may use constant declarations (with **const**).

Answer:

```
const t1 = 3;
const e1 = 2 * t1 + 1;
2 * e1 + 1;
```

Question 5: Lists (3 points)

Which of the following defined expressions (`lst1`, `lst2`, `lst3`) are valid according to the list principle?

```
const lst1 = pair(3, pair(5, pair(10, null)));

const lst2 = pair(pair(3, 5), pair(9, 8));

const lst3 = pair(pair(2, 3), pair(5, null));
```

Answer: `lst1, lst3`

Justify your answer (for all three lists):

The list principle states that a list is either **null** or a pair whose tail is a list. `lst1` satisfies this principle and defines a list [3, 5, 10]. Similarly, `lst3` satisfies the principle and defines a list with two elements, where the first element is a pair. `lst2` is not a list (just a pair of pairs), because it does not have **null** to denote the end of the list.

Question 6: Data abstraction (3 points)

A rectangle consists of an x and a y coordinate, and the width and height of the rectangle. Suppose that we are writing a program that manipulates many rectangles. To make the code easier to read and write, we want to define a rectangle abstraction and a set of functions that manipulate such rectangles.
Your task is to define three functions `make_rectangle`, `is_square` and `at_origin`:

- `make_rectangle` constructs a rectangle from the coordinate, width and height arguments
- `is_square` takes a rectangle constructed by `make_rectangle` as input and checks whether it is square, i.e. both sides have equal length
- `at_origin` takes a rectangle constructed by `make_rectangle` as input and checks whether it is placed at the origin, i.e. the coordinates are both 0.

Here is an example usage of the functions:

```
const rect1 = make_rectangle(1, 4, 5, 5);
const rect2 = make_rectangle(0, 0, 8, 5);
display(is_square(rect1));     // prints 'true'
display(is_square(rect2));     // prints 'false'
display(at_origin(rect1));     // prints 'false'
display(at_origin(rect2));     // prints 'true'
```

Answer:

```
function make_rectangle(x, y, length_x, lenght_y) {
    return list(x, y, length_x, lenght_y);
}

function is_square(rect) {
    return list_ref(rect, 2) === list_ref(rect, 3);
}

function at_origin(rect) {
    return list_ref(rect, 0) === 0 && list_ref(rect, 1) === 0;
}
```

Question 7: List processing (3 points)

Write a function `make_pairs` that takes as input two lists `xs` and `ys` and that constructs a list of the same length as `xs` where each element is a pair of the corresponding elements of `xs` and `ys`. You can assume that `xs` and `ys` have the same length.
For example,

```
make_pairs(list(1,2,3), list(10,20,30));
```

yields the same list as

```
list(pair(1,10), pair(2,20), pair(3,30));
```

Answer:

```
function make_pairs(xs,ys) {
    return is_null(xs)
           ? null
           : pair(pair(head(xs), head(ys)),
                  make_pairs(tail(xs), tail(ys)));
}
```

Question 8: Orders of growth (1 point)

What is the order of growth of the runtime $T(n)$ of the following function in terms of the argument `n`?

```
// n is a non-negative integer
function fun_loops(n) {
    let sum = 0;
    for (let x = 1; x <= n; x = x + 1) {
      sum = sum + 1;
    }
    for (let y = 2 * n; y >= 1; y = y - 1) {
      sum = sum + 1;
    }
    return sum;
}
```

Answer:
$T(\mathsf{n}) \in \Theta(n)$

Question 9: Recurrences from program (3 points)

Recall the interface for creating binary trees seen in class:

```
make_empty_tree(), make_tree(entry, lbranch, rbranch)
```

Give a recurrence that describes the running time $T(\texttt{depth})$ of `make_full_tree` as a function of the `depth` argument (that is assumed to be non-negative).
Use the convention of setting the highest coefficient of polynomials to 1, and lower ones to 0.

```
function make_full_tree(depth, content) {
  return depth === 0
         ? make_tree(content, make_empty_tree(), make_empty_tree())
         : make_tree(content,
                     make_full_tree(depth - 1, content),
                     make_full_tree(depth - 1, content));
}
```

$T(\texttt{depth}) = 2\,T(\texttt{depth} - 1) + 1$

Motivation:

The case where `depth` is 0 takes constant time. When `depth` > 0, the function performs five operations that each takes constant time (comparison, conditional, two subtractions and a call to `make_tree`), and two recursive calls with first argument `depth - 1` that each takes time $T(\texttt{depth} - 1)$. We simplify the constant term to 1.

Question 10: Orders of growth (3 points)

Recall the Muster theorem (for decremental recurrences):
Assume that $T(n) = a\,T(n-b) + f(n)$ for some $a > 0$, $b > 1$, and $f(n) \in \Theta(n^k)$.

1. If $a < 1$ then $T(n) \in \Theta\left(n^k\right)$.
2. If $a = 1$ then $T(n) \in \Theta\left(n^{k+1}\right)$.
3. If $a > 1$ then $T(n) \in \mathrm{O}\left(a^{n/b} n^k\right)$.

Apply this theorem to the recurrence $T(n) = 2\,T(n-1) + 3n$. Show your reasoning, and do not skip steps.

Answer:

We have $a = 2$, $b = 1$, and $f(n) = 3n \in \Theta(n)$ so $k = 1$. Since $a > 1$ case 3 of the Muster theorem applies, and we get that $T(n) \in \mathrm{O}\left(2^n\, n\right)$.

Question 11: Mutable lists (3 points)

Recall the constant **null** and the functions `is_null(list)`, `tail(list)`, and `set_tail(list, new_tail)`.

```
// The argument ls is a list
function mutable_fun(ls) {
    let result = null;
    while (!is_null(ls)) {
        const temp = tail(ls);
        set_tail(ls, result);
        result = ls;
        ls     = temp;
    }
    return result;
}
```

We want to document the function `mutable_fun(list)` above, with a concise description: What side effects does the function have, and what does it return?

HINT: Try the function on lists of length 0, 1, 2, ... until you see a pattern.

Answer: It reverses `ls` in place, and returns the head of the reversed list.

Question 12: Memoization (1 point)

Is the function `mutable_fun` above a good candidate for memoization? Motivate your answer.

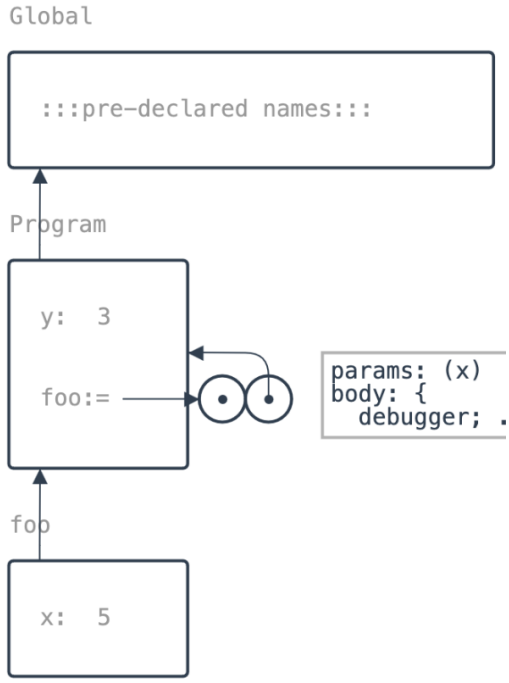
Answer: No, since the function modifies its argument. (The memoized version would only modify the argument the first time that it is used, which would be difficult for other code to handle.)

Question 13: Environment model (2 points)

Recall the environment model from the lectures that explains how mutable Source programs are evaluated. Draw the environment as it looks like when the evaluation of the following program reaches the program location marked with `HERE`:

```
let y = 3;
function foo(x) {
    // HERE
    y = x + 1;
}
foo(5);
```

Answer:



Question 14: Sorting (3 points)

Recall the **Merge sort** sorting algorithm and its immutable (functional) implementation in the `merge_sort` function. Suppose the algorithm is applied to the list `list(5, 3, 7, 9, 1, 4, 8, 2)`. Give the sequence of arguments to the recursive calls of `merge_sort` during the run of the algorithm, i.e. give the arguments of **all** calls to the `merge_sort` function.

Answer:

`list(5, 3, 7, 9), list(1, 4, 8, 2)`

`list(5, 3), list(7, 9), list(1, 4), list(8, 2)`

`list(5), list(3), list(7), list(9), list(1), list(4), list(8), list(2)`

Question 15: Arrays (2 points)

What is the result of running the following program:

```
function count(arr) {
    const length_x = array_length(arr);
    const length_y = array_length(arr[0]);
```

```
    let count = 0;
    for (let i = 0; i < length_x; i = i + 1) {
        for (let j = 0; j < length_y; j = j + 1) {
            if (arr[i][j] === -1) {
                break;
            } else if (arr[i][j] === 0) {
                count = count + 1;
            } else {}
        }
    }
    return count;
}
const grid = [ [1, 0, 1, 0],
               [-1, 0, 1, 0],
               [0, 0, -1, 0]];
count(grid);
```

Answer: 4

Justify your answer:
The function takes as input a 2D array and counts the number of zeros in each row that come before a -1 appears. If a row contains a -1, the counting immediately continues in the next row due to the break statement.

Question 16: Streams (2 points)

Recall the infinite stream `ones` with values $1, 1, 1, 1, \ldots$ defined as follows:

```
const ones = pair(1, () => ones);
```

Define an infinite stream `flip_flop` with values $0, 1, 0, 1, 0, 1, \ldots$.
Try to ensure that the stream uses O(1) memory.

```
const flip_flop = pair(0, () => flop_flip);
const flop_flip = pair(1, () => flip_flop);
```

This is the final page.